



PEC3 - Tercera prueba de evaluación continua

Presentación

Esta PEC se focaliza en los circuitos secuenciales. Los circuitos combinacionales nos permiten describir funcionalidades de un circuito, pero no nos permiten guardar información. Mediante biestables y registros podemos guardar información en memoria y hacer circuitos más complejos. En esta PEC practicaremos con este tipo de circuitos.

Competencias

- Entender el funcionamiento de los circuitos lógicos secuenciales y conocer y saber aplicar técnicas de diseño de sistemas secuenciales.

Objetivos

- Saber discernir, a partir de la funcionalidad que se quiere que tenga un circuito lógico, si el circuito tiene que ser de tipo secuencial o combinacional.
- Conocer el funcionamiento del biestable D y de todas las entradas de control que puede tener.
- Saber analizar un circuito secuencial.
- Saber realizar un cronograma a partir de un circuito digital secuencial.
- Saber analizar un grafo de estados.
- Saber diseñar un circuito cualquiera a partir de la descripción de su funcionalidad mediante el modelo de Moore.

Recursos

Los recursos que se recomienda usar para esta PEC son los siguientes:

Básicos: El módulo 4 de los materiales.

Complementarios: VerilCIRC, VerilCHART y el Wiki de la asignatura.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.
- La valoración está indicada en cada uno de los subapartados.

Formato y fecha de entrega



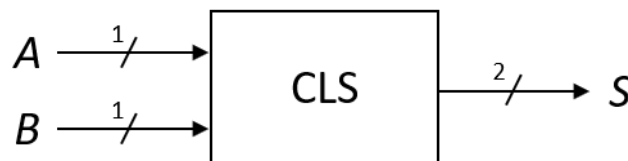


- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF usando una de las plantillas entregadas juntamente con este enunciado.
- Se tiene que entregar a través de la aplicación de **Entrega y registro de EC** del apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **24 de noviembre** (a las 24 horas).

Solución de la PEC

Ejercicio 1 [25%]

Se quiere diseñar el grafo de estados de un circuito lógico secuencial (CLS) con dos entradas, A y B , de un bit cada una, y una salida S de dos bits. El circuito tiene la estructura siguiente:



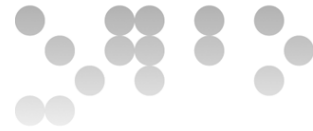
El funcionamiento del circuito es el siguiente. El circuito debe ir leyendo bits de las dos entradas hasta que detecte un cero, tanto en A como en B , durante dos ciclos consecutivos. A partir de ese momento, por cada entrada, se leerá un valor en secuencia durante dos ciclos (en el primer ciclo tendremos el bit de más peso y en el segundo ciclo el de menos peso). Una vez leídos los dos valores de dos bits, el circuito indicará en el ciclo siguiente por qué entrada se ha introducido el valor máximo: si el valor introducido por la entrada A es más grande que el de la entrada B , entonces la salida S será 10 durante un ciclo; si el valor de B es más grande que el de A , la salida S será 01; finalmente si los valores de A y B son iguales entonces la salida S será 11 durante un ciclo. Durante el resto de los ciclos la salida será siempre 00.

Una vez detectada y analizada una secuencia (dos ceros por las dos entradas seguido de dos bits), el circuito vuelve a buscar la siguiente secuencia.

Ejemplo de funcionamiento:

Entrada A	0	0	0	0	0	1	0	1	0	0	1	0	1	...
Entrada B	0	1	0	0	1	0	0	0	0	0	0	0	1	...
Salida S	00	00	00	00	00	00	01	00	00	00	00	00	10	00

Fijaos que en los ciclos 3-4 se detectan los dos ciclos de ceros (en negrita) y, a partir de ese instante, se lee por A el valor 01 y por B el valor 10, así que en el siguiente ciclo S valdrá 01 (en negrita). Similarmente, en los ciclos 9-10 se detectan dos ciclos



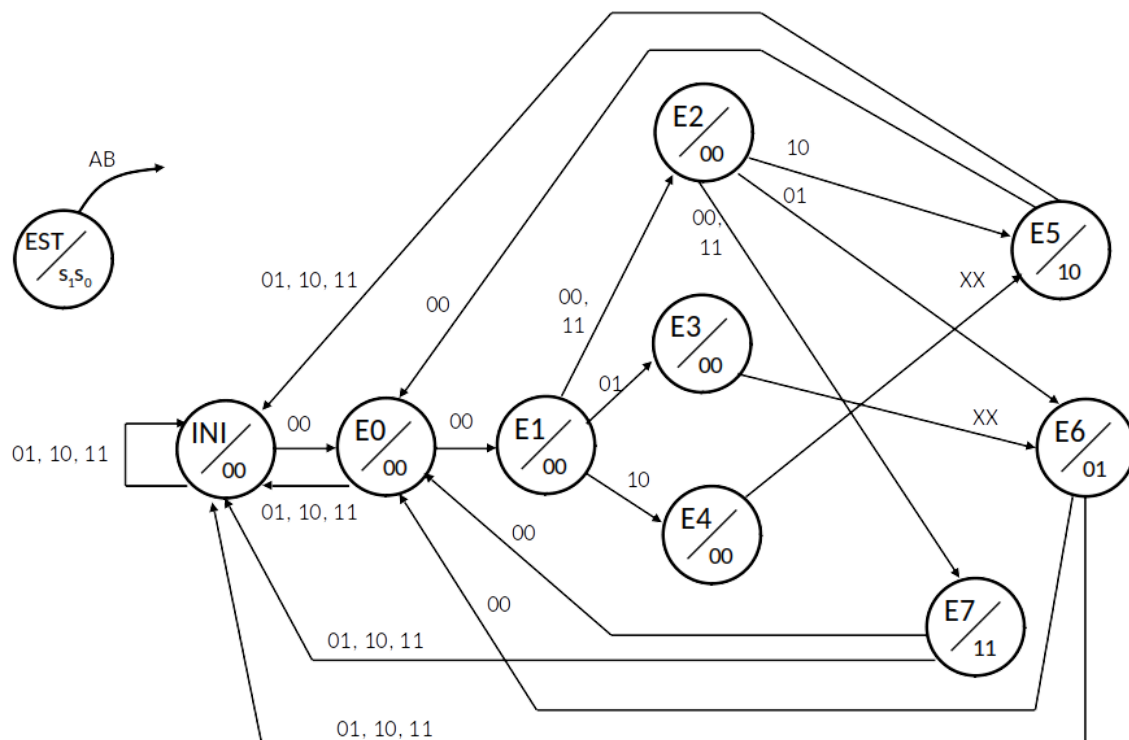
de ceros (también en negrita) y, a partir de ese instante se lee por *A* el valor 10 y por *B* el valor 00, así que en el siguiente ciclo *S* valdrá 10.

Se pide que diseñéis el grafo de estados del circuito CLS, especificando claramente la leyenda del mismo.

Para conseguir el funcionamiento deseado, el circuito debe tener los siguientes estados:

Estado	Descripción	Salida
<i>INI</i>	Estado inicial, no se ha reconocido nada de la secuencia	00
<i>E0</i>	Se ha visto la entrada <i>AB</i> = 00	00
<i>E1</i>	Se ha visto la entrada <i>AB</i> = 00 (se han visto las entradas 00 en dos ciclos consecutivos)	00
<i>E2</i>	Se ha visto la entrada <i>AB</i> = 00 o <i>AB</i> = 11	00
<i>E3</i>	Se ha visto la entrada <i>AB</i> = 01	00
<i>E4</i>	Se ha visto la entrada <i>AB</i> = 10	00
<i>E5</i>	El valor de <i>A</i> en dos ciclos consecutivos es mayor que <i>B</i> ($A > B$)	10
<i>E6</i>	El valor de <i>A</i> en dos ciclos consecutivos es menor que <i>B</i> ($A < B$)	01
<i>E7</i>	El valor de <i>A</i> en dos ciclos consecutivos es igual que <i>B</i> ($A = B$)	11

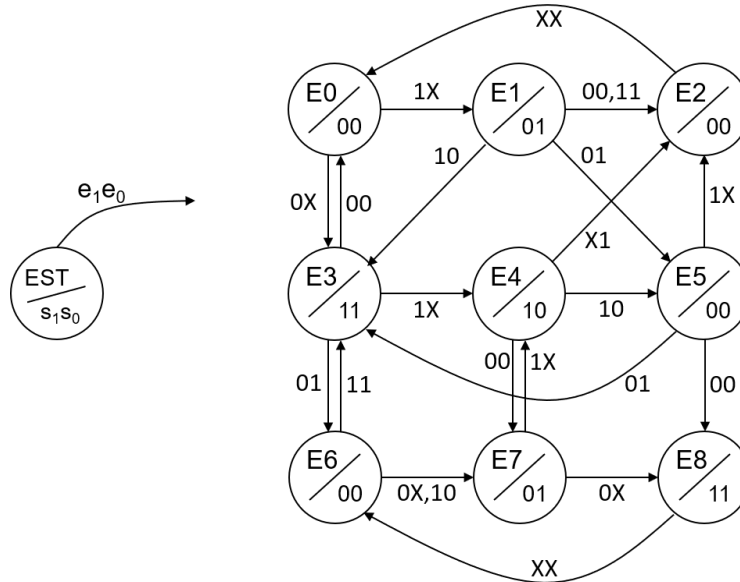
El siguiente grafo representa el comportamiento deseado:





Ejercicio 2 [25%]

Dado el grafo de estados siguiente:



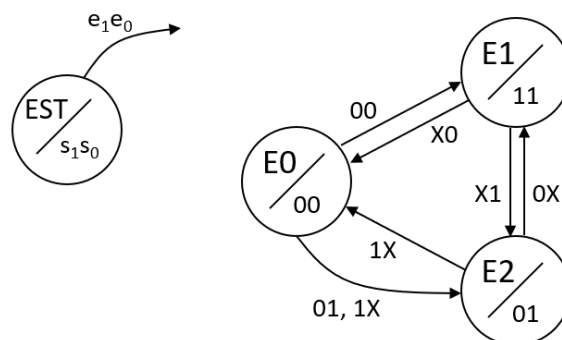
Se pide:

- a) [5%] ¿Cuántos bits de entrada tiene el circuito que implementa este grafo? ¿Cuántos bits de salida? ¿Cuál será el número mínimo de biestables para implementarlo? Si lo implementamos usando una memoria ROM, ¿cuántos bits de direcciones y cuántos bits de datos necesitará esta memoria?

El circuito tiene dos bits de entrada, denominados e_1 y e_0 . Los bits de salida son dos, denominados s_1 y s_0 . Como el circuito tiene 9 estados necesitamos 4 bits para representarlos y, por lo tanto, necesitamos 4 biestables para poder almacenar estos 4 bits.

La memoria ROM que puede implementar este circuito tendría 6 bits de direccionamiento, 4 bits para el estado más 2 bits para la entrada. El ancho de las palabras de la ROM tendría que ser de 6 bits, 2 bits para guardar la salida y 4 bits para codificar el estado futuro.

Dado el grafo de estados siguiente:





Se pide:

- b) **[10%]** Rellenad la tabla de transiciones y salidas siguiente (fijaos que se han unificado en una única tabla de excitaciones), codificando los estados según su índice asociado.

q_1	q_0	e_1	e_0	q_1^+	q_0^+	s_1	s_0

Nota: Tenéis disponible el ejercicio en VerilCHART. Para poder probar este ejercicio en VerilCHART tenéis que reemplazar, si fuera el caso, los bits *don't care* por valores 0.

La siguiente tabla establece para las columnas q_1+q_0+ cuál es el estado en el ciclo siguiente a partir del valor que toman las señales de entrada. Las columnas de las salidas s_1, s_0 determinan el valor de las salidas para cada estado, teniendo en cuenta que las salidas dependen del estado, no del estado futuro.

q_1	q_0	e_1	e_0	q_1^+	q_0^+	s_1	s_0
0	0	0	0	0	1	0	0
0	0	0	1	1	0	0	0
0	0	1	0	1	0	0	0
0	0	1	1	1	0	0	0
0	1	0	0	0	0	1	1



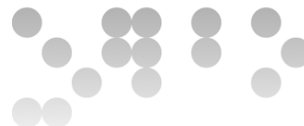
0	1	0	1	1	0	1	1
0	1	1	0	0	0	1	1
0	1	1	1	1	0	1	1
1	0	0	0	0	1	0	1
1	0	0	1	0	1	0	1
1	0	1	0	0	0	0	1
1	0	1	1	0	0	0	1
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

- c) **[5%]** Si implementamos el circuito representado por la tabla de salidas y excitaciones del apartado anterior usando una memoria ROM, especificad en binario y en hexadecimal el contenido de todas las posiciones de la memoria ROM.

Nota: Tenéis disponible el ejercicio en VerilCIRC. Para poder probar este ejercicio en VerilCIRC tenéis que reemplazar, si fuera el caso, los bits *don't care* por valores 0.

El contenido de la memoria ROM corresponde a las columnas $q_1+q_0+s_1s_0$ de la tabla del apartado anterior.

@	binario	hex
0	0100	4
1	1000	8
2	1000	8
3	1000	8
4	0011	3
5	1011	B
6	0011	3
7	1011	B
8	0101	5
9	0101	5
10	0001	1
11	0001	1
12	x	x
13	x	x



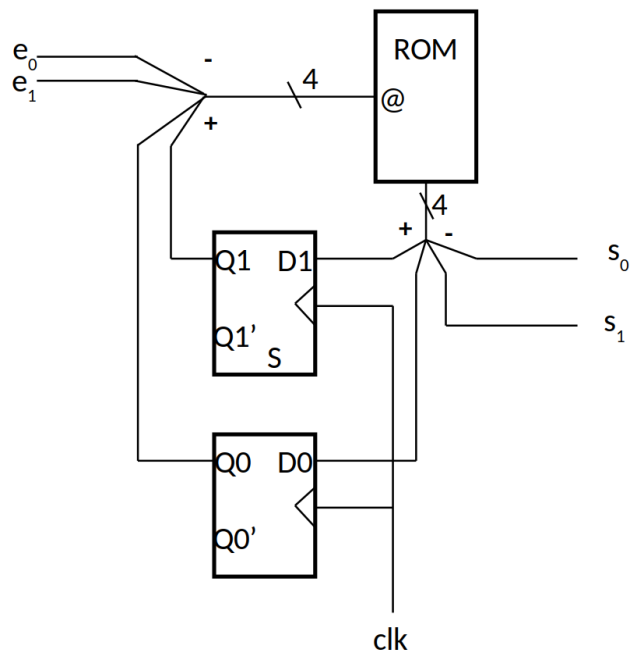
14	x	x
15	x	x



- d) **[5%]** Implementad el circuito representado por el grafo usando la memoria ROM propuesta en el apartado anterior.

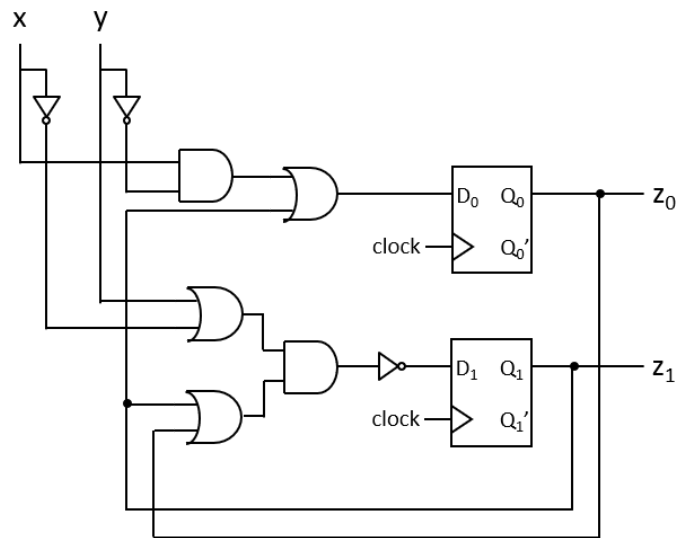
Nota: Tenéis disponible el ejercicio en VerilCIRC. Para poder probar este ejercicio en VerilCIRC tenéis que reemplazar, si fuera el caso, los bits *don't care* por valores 0.

Implementamos la tabla obtenida en el apartado b) con el siguiente circuito de dos biestables y una ROM, donde el contenido de la ROM es el del apartado anterior.



Ejercicio 3 [25%]

Dado el circuito siguiente:



Analizad el circuito y rellenad la tabla de transiciones y salidas siguiente, incluyendo los valores intermedios mostrados en la tabla:

q_1	q_0	x	y	d_1	d_0	z_1	z_0

Nota: Tenéis disponible las tablas de forma unificada en VerilCHART.

El circuito se puede describir con las siguientes expresiones algebraicas que usamos para obtener los valores de la tabla:



$$d_1 = ((x' + y)(q_1 + q_0))'$$

$$d_0 = xy' + q_1$$

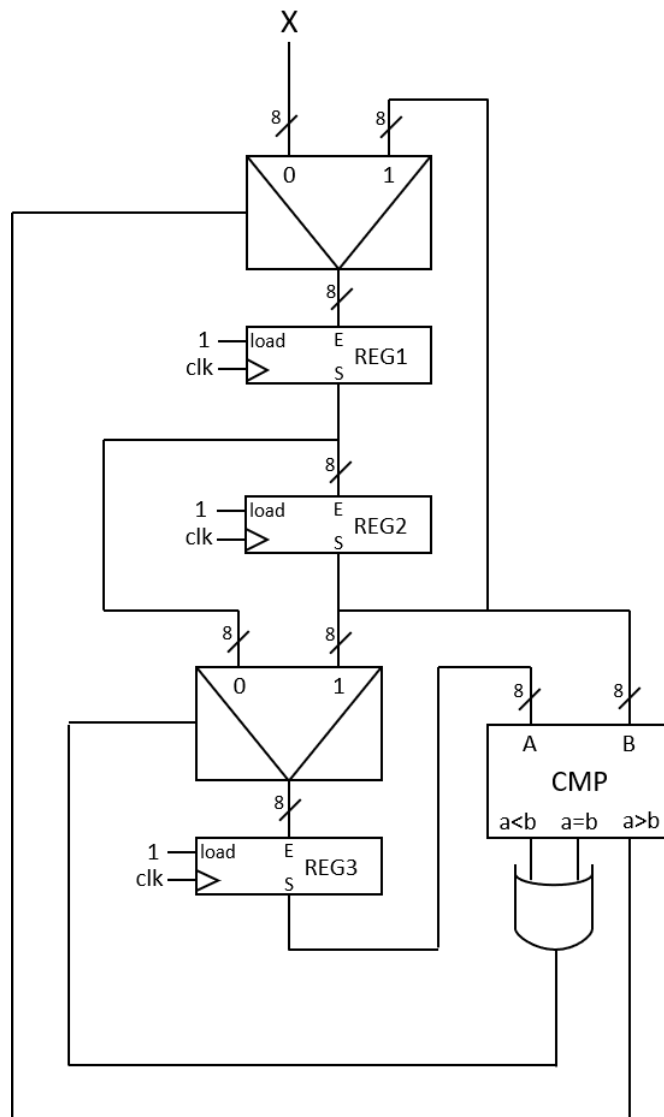
$$z_1 = q_1$$

$$z_0 = q_0$$

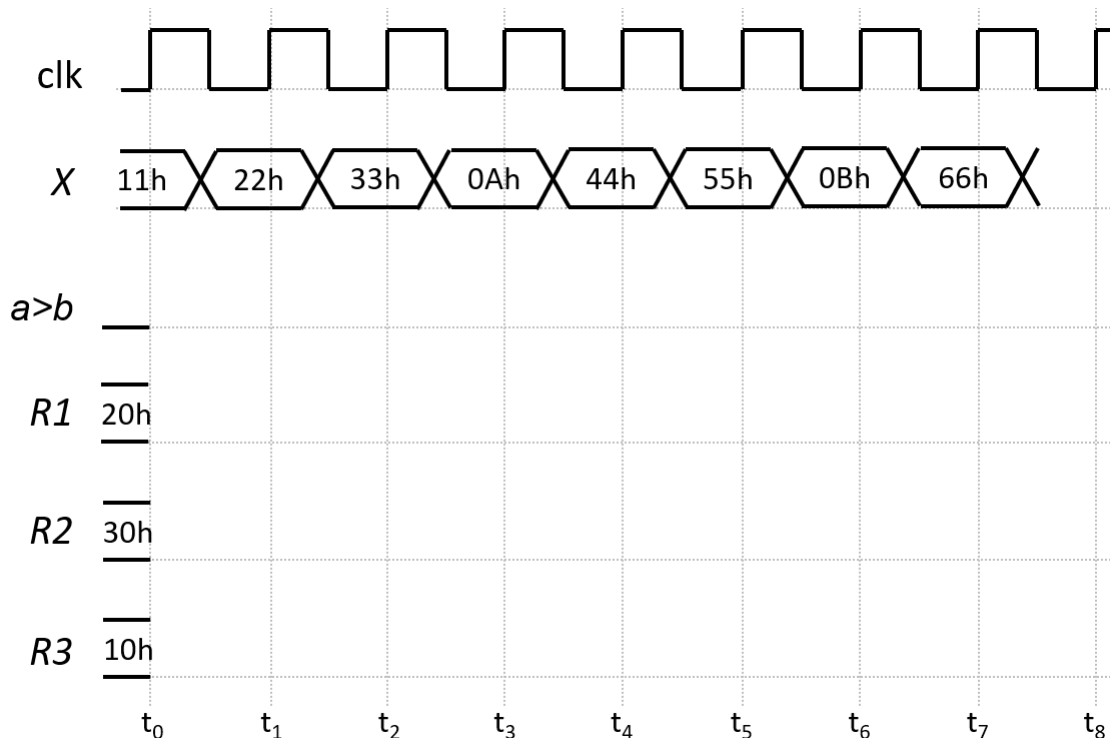
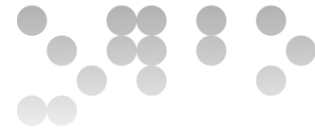
q_1	q_0	x	y	d_1	d_0	z_1	z_0
0	0	0	0	1	0	0	0
0	0	0	1	1	0	0	0
0	0	1	0	1	1	0	0
0	0	1	1	1	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	0	1
0	1	1	0	1	1	0	1
0	1	1	1	0	0	0	1
1	0	0	0	0	1	1	0
1	0	0	1	0	1	1	0
1	0	1	0	1	1	1	0
1	0	1	1	0	1	1	0
1	1	0	0	0	1	1	1
1	1	0	1	0	1	1	1
1	1	1	0	1	1	1	1
1	1	1	1	0	1	1	1

Ejercicio 4 [25%]

Dado el circuito siguiente:



Completad el cronograma siguiente, poniendo los valores de los registros en hexadecimal.



Nota: Tenéis disponible este ejercicio en VerilCHART.

Señal $a > b$: La salida del comparador se activa cuando la entrada a (salida del registro R3) es mayor que la entrada b (salida del registro R2).

Señal $R1$: Corresponde a la salida del registro R1. En cada flanco ascendente de reloj se carga un nuevo contenido al registro, debido a que $load = 1$. El valor que se carga depende del valor de la señal $a > b$, que determina el valor de la salida del multiplexor.

Señal $R2$: Corresponde a la salida del registro R2. En cada flanco ascendente de reloj se carga un nuevo contenido al registro, debido a que $load = 1$. El nuevo contenido es siempre el valor de la salida del registro R1.

Señal $R3$: Corresponde a la salida del registro R3. En cada flanco ascendente de reloj se carga un nuevo contenido al registro, debido a que $load = 1$. El valor que se carga depende del valor de la señal $a < b$, que determina el valor de la salida del multiplexor.

Señal $a \leq b$: La salida del comparador se activa cuando la entrada a (salida del registro R3) es menor o igual que la entrada b (el valor de la salida del registro R2).

Con estas especificaciones generales el cronograma queda:

